AL-ZAYTOONAH UNIVERSITY OF JORDAN

LECTURE 2 · BLOCK 2: KNOW WHERE YOU ARE

ROBOT LOCALIZATION AND STATE ESTIMATION

Introduction to Robotics for AI and Data Science

Dr. Akram Fatayer

a.fatayer@zuj.edu.jo

linkedin.com/in/akram-fatayer/

Integration Map: You Are Here



Block 2: Know Where You Are

Before your robot can plan a path, it must know where it is.

The Fundamental Problem: Where Am I?

A robot never knows exactly where it is. Two things are always uncertain:



Its motion

Wheels slip and floors are uneven: after a move, the robot is only roughly where it planned to be.

→ the predict step



Its sensing

Every sensor reading is noisy, and some sensors are slow: no reading tells the exact truth.

→ the update step

So instead of one position, the robot keeps a probability distribution.

How do we reason with uncertain motion and uncertain sensing?

A Belief Is a Probability Distribution

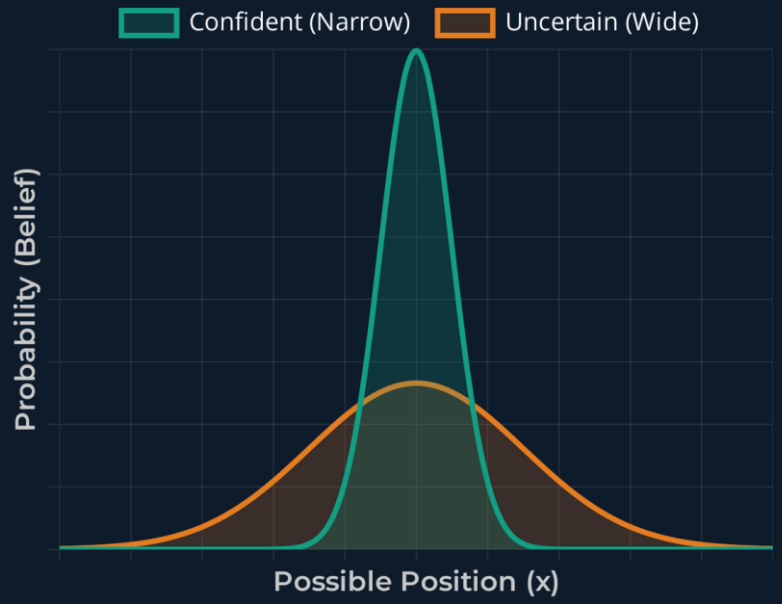
A robot never knows its exact position. It keeps a **belief**: a probability distribution over where it might be.

The Kalman filter uses a Gaussian, described by two numbers:

- μ mean: the best guess of the position
- σ standard deviation: how unsure the robot is
- Narrow curve: confident. Wide curve: lost.

$$p(x) = 1 / (\sigma\sqrt{(2\pi)}) \cdot \exp(-(x - \mu)^2 / (2\sigma^2))$$

About 95% of the belief lies within **μ ± 2σ**: the shaded band in the Lab 3 plots. The variance σ^2 is what the filter tracks; it calls it P.



Probabilistic Robotics (Thrun, Burgard and Fox, 2005)

Motion Alone: The Predict Step

$$\hat{x}^{-} = F \cdot x + B \cdot u$$

$$P^{-} = F \cdot P \cdot F^{T} + Q$$

F	How the state evolves on its own (here 1)
$B \cdot u$	Odometry's report: distance moved since the last step
P	Variance of the estimate (uncertainty², m²)
Q	Odometry's error per step: slip, bias, noise

Some books, and Lab 3 Activity 2, treat odometry as a velocity measurement in the update step instead. The result is almost the same.

1D EXAMPLE

The robot is at x = 1 m with P = 0.25 m². Odometry reports it moved u = 1 m; Q = 0.1 m².

Here F = 1 and B = 1, so:

$$\hat{x}^{-} = 1 + 1 = 2 \text{ m}$$

$$P^{-} = 0.25 + 0.1 = 0.35 \text{ m}^2$$

Odometry alone: P only grows. Over five laps of a 5 m circle, the error reaches 4.9 m: it drifts.

Sensing Alone: GPS

GPS

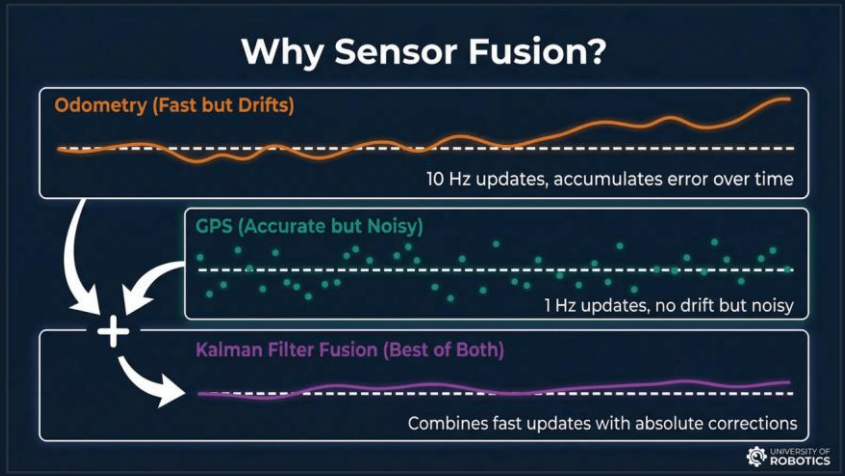
Measures the absolute position.
It never drifts.

Noisy and slow: about 3 m error, once a second.

In the filter it is a measurement:

$$z = \text{position},\ H = 1,\ R = 3^2 = 9\ \text{m}^2$$

GPS alone (hold the last fix): 4.4 m error, and the estimate jumps every second.



Odometry is smooth but drifts; GPS is noisy but never drifts. They fail in opposite ways.

First Idea: The Gated Reset

A natural first attempt: trust odometry, and let GPS stop the drift.

1. Follow odometry between GPS fixes.
2. When a GPS fix arrives, compare it with the odometry position.
3. Within the gate ($2\sigma = 6$ m)? Keep odometry. Outside? Jump to the GPS fix and restart odometry from there.

Simple, and the drift can no longer grow forever.

RESULT: 5 LAPS, GPS $\sigma = 3$ m, 20 RUNS

Odometry only	4.85 m
GPS only	4.43 m
Gated reset	4.05 m

Largest jump in one 0.1 s step	12.7 m

(the robot really moves 0.26 m per step)

Better than either sensor, but only a little. Why?

Why the Reset Fails

1 **It copies the GPS error**

Right after a reset you are exactly as wrong as that one GPS fix, about 3 m. You can never beat GPS.

2 **It accepts drift up to the gate**

Between resets, odometry may wander up to 6 m before anything happens.

3 **It throws information away**

Fixes inside the gate are ignored; at each reset the odometry history is lost. Averaging many fixes would cancel their noise.

4 **Noise triggers false resets**

One unlucky GPS fix can make good odometry jump: the 12.7 m jumps.

The problem is all-or-nothing. What if each GPS fix moved the estimate only part of the way?

Better Idea: Weigh by Uncertainty

Two GPS units measure the same position. Trust each in proportion to $1 / \sigma^2$.

GPS A reads 10 m (σ = 1 m)
GPS B reads 12 m (σ = 2 m)

$$x = (10/1 + 12/4) \,/\, (1/1 + 1/4) = 10.4 \text{ m}$$

$$\sigma = 1 \,/\, \sqrt{(1 + 1/4)} = 0.89 \text{ m}$$

More certain than either unit alone.

THE SAME THING AS A CORRECTION

Start from A ($P = 1 \text{ m}^2$) and correct towards B ($R = 4 \text{ m}^2$):

$$K = P \,/\, (P + R) = 1 \,/\, (1 + 4) = 0.2$$

$$x = 10 + 0.2 \times (12 - 10) = 10.4 \text{ m}$$

K is the Kalman gain.

A soft reset: the gated reset uses K = 0 or K = 1. The Kalman filter uses the K that the uncertainties call for.

The Update Step

$$K = P^{-} \cdot H^{T} \cdot (H \cdot P^{-} \cdot H^{T} + R)^{-1}$$

$$\hat{x} = \hat{x}^{-} + K \cdot (z - H \cdot \hat{x}^{-})$$

$$P = (I - K \cdot H) \cdot P^{-}$$

z	The measurement: what the sensor reads
H	Which part of the state the sensor measures
R	Measurement noise: the sensor's variance
$z - H \cdot \hat{x}^{-}$	Innovation: reading minus prediction
K	Kalman gain: 0 trusts the prediction, 1 the sensor

1D EXAMPLE (H = 1)

Prediction $\hat{x}^{-} = 4$ m, $P^{-} = 1.5^2 = 2.25$ m². GPS reads $z = 6.5$ m with $R = 1$ m².

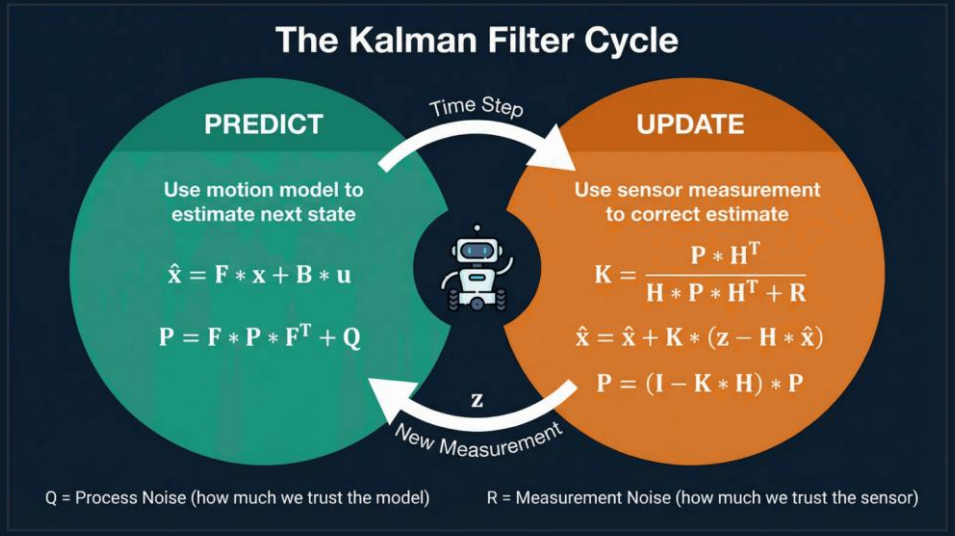
$$K = 2.25 \,/\, (2.25 + 1) = 0.69$$

$$\hat{x} = 4 + 0.69 \times (6.5 - 4) = 5.73 \text{ m}$$

$$P = (1 - 0.69) \times 2.25 = 0.69 \text{ m}^2$$

σ = 0.83 m: smaller than both the prediction (1.5 m) and GPS (1 m).

Predict + Update over Time: The Kalman Filter



Every step: predict with odometry. P grows.

Every GPS fix: update. P shrinks.

K is small while odometry is fresh, and grows with P: the longer since the last fix, the more GPS counts.

Odometry only	4.85 m
GPS only	4.43 m
Gated reset	4.05 m
Kalman filter	1.35 m

Sensor Fusion vs the Kalman Filter

Sensor fusion is the goal: combine several sensors into one estimate better than any of them.

The Kalman filter is one method: the best one when the models are linear and the noise is Gaussian.

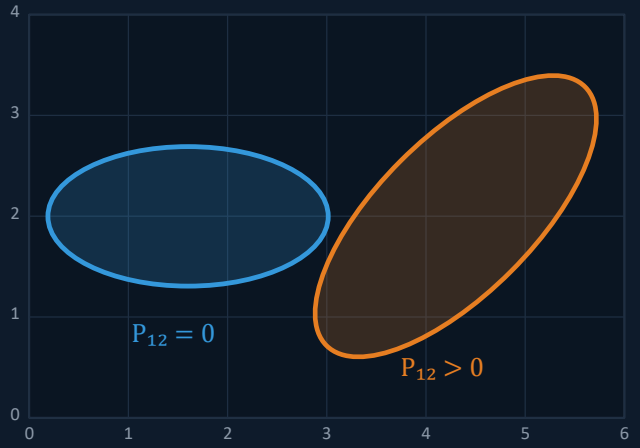
	Uses a Kalman filter	No Kalman filter
Fuses sensors	Lab 3 Activity 3: odometry + GPS	Two GPS units averaged by $1/\sigma^2$; Lecture 4's occupancy grid fuses LiDAR beams with Bayes' rule
One sensor only	Lab 3 Activity 1: one noisy sensor + a motion model	Dead reckoning: just add up odometry

Other fusion methods: complementary filters (common for IMUs), extended and unscented Kalman filters (Lecture 3), particle filters, learned fusion.

From Numbers to Matrices

In 2D the equations stay exactly the same. Every number becomes a vector or a matrix.

	1D (one number)	2D (the robot in the plane)
State	x	$x = [x, y]^T$ a column vector
Uncertainty	P a variance	$P = [[P_{11}, P_{12}], [P_{21}, P_{22}]]$ a covariance matrix
Gain	$K = P / (P + R)$	$K = P \cdot H^T \cdot (H \cdot P \cdot H^T + R)^{-1}$
Picture	a bell curve	an ellipse



Diagonal P_{11}, P_{22}: the variances in x and in y.

Off-diagonal $P_{12} = P_{21}$: how the errors in x and y move together. It tilts the ellipse.

Lab 3 Activity 2 tracks $x = [x, y, vx, vy]^T$: the same equations with 4×4 matrices.

The Matrix Toolkit You Need

Add: entry by entry

$$\begin{bmatrix} 1.5 & 0.75 \\ 0.75 & 1.5 \end{bmatrix} + \begin{bmatrix} 0.5 & 0.25 \\ 0.25 & 0.5 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Used in $P^- = P + Q$ and $S = P^- + R$

Multiply: row × column

$$\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ -0.5 \end{bmatrix} = \begin{bmatrix} 1.5 \\ 0 \end{bmatrix}$$

Each entry: a row on the left times a column on the right

Transpose: rows become columns

$$[1 \quad 0]^T = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$H = [1, 0]$ gives $H^T = [1, 0]^T$, a column

Inverse of a 2×2 matrix

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = 1/(ad - bc) \cdot \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

Divide by det = ad − bc; swap a and d; negate b and c

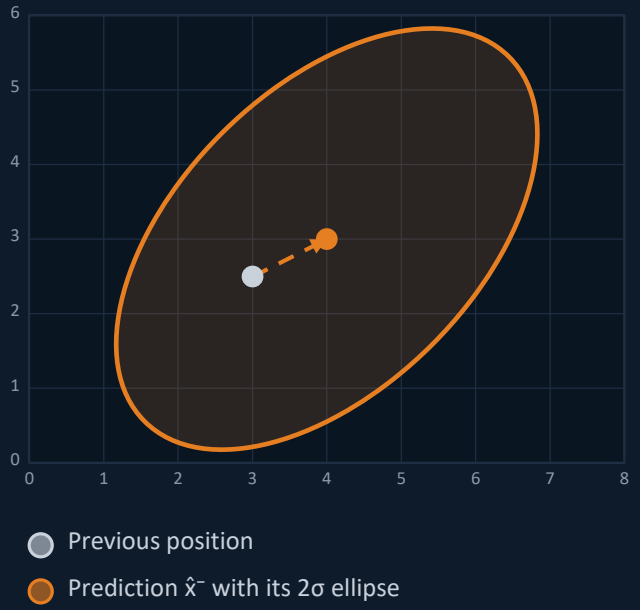
2D Example, Step 1: Predict

The robot was at [3, 2.5] m. Odometry reports it moved u = [1, 0.5] m since the last step, so F = I and B = I.

$$\hat{x}^{-} = F \cdot x + B \cdot u = \begin{bmatrix} 3 \\ 2.5 \end{bmatrix} + \begin{bmatrix} 1 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

$$P^{-} = F \cdot P \cdot F^{T} + Q = \begin{bmatrix} 1.5 & 0.75 \\ 0.75 & 1.5 \end{bmatrix} + \begin{bmatrix} 0.5 & 0.25 \\ 0.25 & 0.5 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

With F = I, $F \cdot P \cdot F^{T}$ is just P. Q has an off-diagonal term because wheel slip happens mostly along the direction of travel, so the uncertainty grows along it.



2D Example, Step 2: The Kalman Gain

A GPS fix arrives: z = [5, 2.5] m. GPS measures both x and y, so H = I, with R = I (σ = 1 m in each axis).

$$S = H \cdot P^{-} \cdot H^{T} + R = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \quad \det S = 3 \cdot 3 - 1 \cdot 1 = 8$$

$$S^{-1} = 1/8 \cdot \begin{bmatrix} 3 & -1 \\ -1 & 3 \end{bmatrix} = \begin{bmatrix} 0.375 & -0.125 \\ -0.125 & 0.375 \end{bmatrix}$$

$$K = P^{-} \cdot H^{T} \cdot S^{-1} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 0.375 & -0.125 \\ -0.125 & 0.375 \end{bmatrix} = \begin{bmatrix} 0.625 & 0.125 \\ 0.125 & 0.625 \end{bmatrix}$$

Check the top-left entry: 2 × 0.375 + 1 × (−0.125) = 0.625. The diagonal of K says how much to trust GPS in x and in y; the off-diagonal lets a correction in one axis move the other.

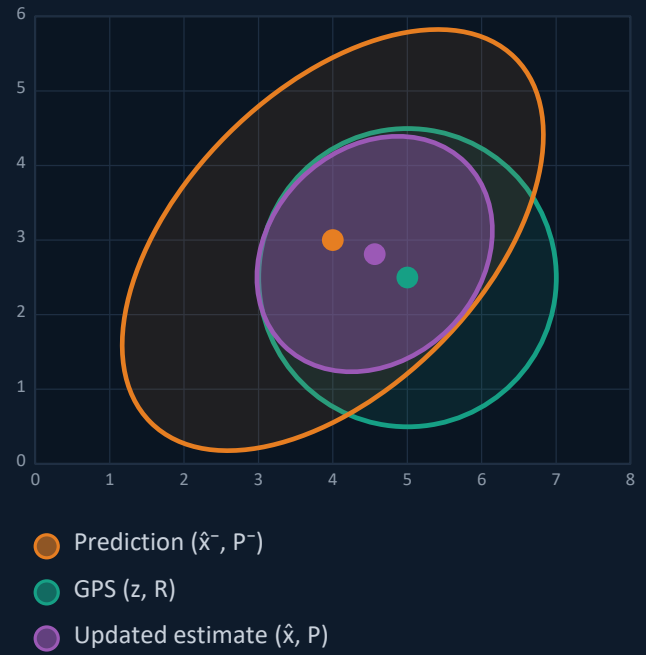
2D Example, Step 3: The New Estimate

$$z - H \cdot \hat{x}^{-} = \begin{bmatrix} 5 \\ 2.5 \end{bmatrix} - \begin{bmatrix} 4 \\ 3 \end{bmatrix} = \begin{bmatrix} 1 \\ -0.5 \end{bmatrix}$$

$$\hat{x} = \begin{bmatrix} 4 \\ 3 \end{bmatrix} + \begin{bmatrix} 0.625 & 0.125 \\ 0.125 & 0.625 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ -0.5 \end{bmatrix} = \begin{bmatrix} 4.56 \\ 2.81 \end{bmatrix}$$

$$P = (I - K \cdot H) \cdot P^{-} = \begin{bmatrix} 0.375 & -0.125 \\ -0.125 & 0.375 \end{bmatrix} \cdot \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 0.625 & 0.125 \\ 0.125 & 0.625 \end{bmatrix}$$

The estimate moved most of the way towards the GPS reading, and the new ellipse is smaller than both the prediction and the GPS ellipse.



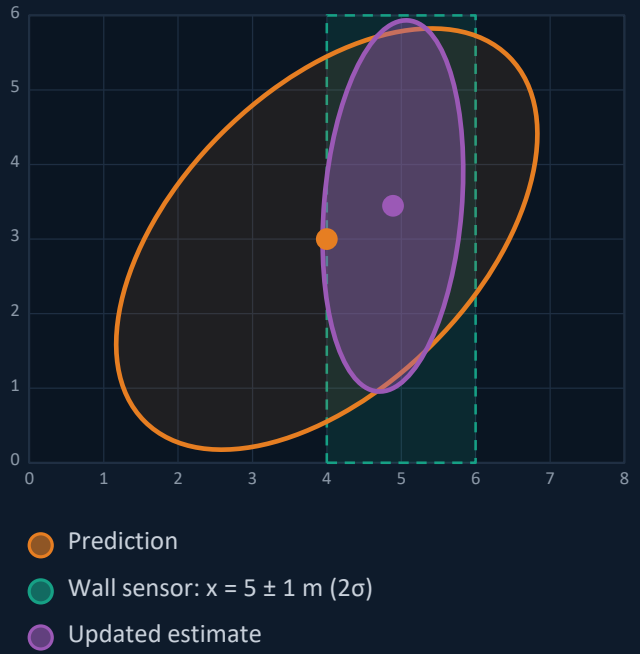
What If the Sensor Measures Only x?

A sensor on the wall measures only x: z = 5 m, with R = 0.25 m². Now H = [1, 0], so S is a single number.

$$S = H \cdot P^{-} \cdot H^{T} + R = P_{11} + R = 2 + 0.25 = 2.25$$

$$K = P^{-} \cdot H^{T} / S = \begin{bmatrix} 2 \\ 1 \end{bmatrix} / 2.25 = \begin{bmatrix} 0.889 \\ 0.444 \end{bmatrix}$$

$$\hat{x} = \begin{bmatrix} 4 \\ 3 \end{bmatrix} + \begin{bmatrix} 0.889 \\ 0.444 \end{bmatrix} \cdot (5 - 4) = \begin{bmatrix} 4.89 \\ 3.44 \end{bmatrix}$$



y moved by 0.44 m although only x was measured. K has a second entry because $P^{-}_{12} = 1 \neq 0$: the filter knows the x and y errors move together. With $P^{-}_{12} = 0$, y would not change.

Tuning Q and R: Whom Do You Trust?

Small Q: trust the motion model

Smooth estimate that ignores noise, but slow to notice when the motion really changes.

Large Q: distrust the model

Follows the measurements closely, noise and all.

Small R: trust the sensor

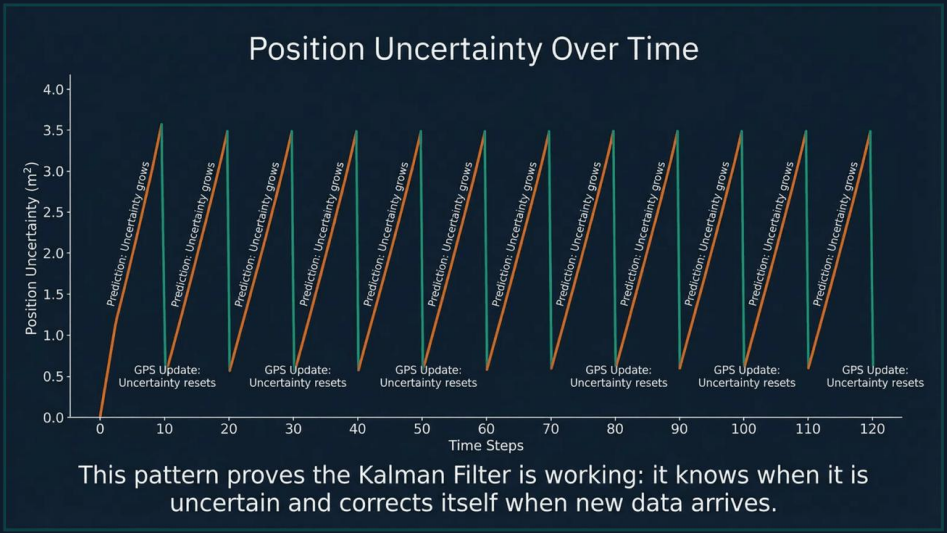
Each reading pulls the estimate hard (K close to 1).

Large R: distrust the sensor

Readings barely move the estimate (K close to 0).

Lab 3, Activity 1: on a steady train a small Q works best. On a train that stops at a station, the same small Q makes the filter overshoot the stop for many steps. There is no single best Q: it depends on how the robot really moves.

The Uncertainty Sawtooth



Predict: P climbs

Each step adds Q: odometry's error builds up between GPS fixes.

GPS update: P drops

Each GPS fix pulls the uncertainty down at once: the sharp drops.

It settles

After a few fixes, every tooth has the same peak and low. They depend only on Q, R and the GPS rate.

Lab connection: you will plot this in Activity 3 and explain its shape.

From Theory to Your Robot: Lab 3



Activity 1: Pure Math

1D Kalman filter in NumPy: the predict and update equations, on a train.



Activity 2: The Robot

2D Kalman filter on the R2D2 robot in PyBullet, tracking an L-shaped path.



Activity 3: Sensor Fusion

Compare odometry, GPS, the gated reset and the Kalman filter, and explain the sawtooth.

PRACTISE BEFORE THE LAB

Interactive module 2, Know Where You Are: the same examples as this lecture, with eight challenges and a completion code.

akramfatayer.github.io/courses/robotics/learn/2

Key Takeaways

Uncertainty is inevitable. A robot keeps a belief, a mean and a variance, not a single position.

Predict, then update. Odometry moves the belief and widens it; GPS corrects it and narrows it.

Weigh, don't switch. The gated reset's all-or-nothing gives 4.05 m; weighting by uncertainty (K) gives 1.35 m.

Matrices change nothing but the bookkeeping. In 2D, P is a covariance matrix; its off-diagonal lets one measurement improve another axis.

NEXT LECTURE

Lecture 3: Motion Models

Real robots turn. What happens when the motion is not a straight line?

Questions & Discussion

Thank you for your attention.

Dr. Akram Fatayer

a.fatayer@zuj.edu.jo

linkedin.com/in/akram-fatayer/